# Distributed In-Memory Key-Value Store (Redis Clone in Java)

Complete Interview & Project Guide (in Hinglish) | Tech Stack: Java 23/24, Spring Boot, Core TCP Sockets

## 1. Project Ka 1-Minute Elevator Pitch (Interview ke liye)

**Pitch:** *"Maine Java aur Spring Boot ka use karke ek Custom In-Memory Key-Value Database banaya hai jo Redis ke core architecture aur RESP v2 (Redis Serialization Protocol) ko implement karta hai. Isme raw TCP sockets aur multithreading ka use kiya hai jo concurrent clients ko serve karta hai. Iske teen main pillars hain: 1. In-Memory Store with TTL/Lazy Expiration, 2. Master-Slave Distributed Replication with 4-step handshake, RDB transfer, command stream propagation, aur WAIT synchronous replication with byte-level offset tracking, 3. ACID Transaction Engine (MULTI, EXEC, DISCARD) jo commands ko FIFO queue karke isolated scratchpad cache me atomically commit karta hai."*

## 2. Project Output Kaise Dikhayein & Explain Karein

Yeh ek background network database engine hai. Iska demo aise diya jata hai:

| Feature | Client Input | Server Output & Explanation |
|---|---|---|
| **Ping Check** | PING | +PONG (Server healthy and responding) |
| **Set / Get** | SET key "val"<br>GET key | +OK<br>"val" (Stored in ConcurrentHashMap) |
| **Counter INCR** | INCR counter | :1, :2, :3... (Thread-safe integer increment) |
| **TTL Expiry** | SET otp 1234 px 1000<br>GET otp (after 1s) | +OK<br>$-1 (nil - Lazy eviction automatically cleans expired key) |
| **Replication** | Master: SET city Delhi<br>Slave: GET city | +OK<br>"Delhi" (Real-time replication stream sync) |
| **Transactions** | MULTI -> SET a 1 -> EXEC | +OK -> +QUEUED -> 1) +OK (Atomic isolated commit) |

## 3. Top 5 Interview Questions & Expert Hinglish Answers

### *Q1: Redis single-threaded hota hai, aapne Java me multithreading kyu use ki?*

**Ans:** Redis C me event loop (Linux epoll) use karta hai. Java me blocking sockets use karte waqt agar single-threaded rehte toh ek slow client poore server ko freeze kar deta. Isliye maine CompletableFuture se har client ko asynchronous thread allocate kiya aur shared store ko ConcurrentHashMap aur ReentrantReadWriteLock se thread-safe banaya.

### *Q2: Master-Slave replication me offset tracking kyu zaroori hai?*

**Ans:** Replication consistency ensure karne ke liye byte-level offset use hota hai. Master ke commands ka raw byte count slave add karta hai. Master jab WAIT command me REPLCONF GETACK * bhejta hai, slave exact ACK reply deta hai, jisse master jaan sake kitne replicas caught-up hain.

### *Q3: Project ka sabse tricky bug kya tha aur kaise solve kiya?*

**Ans:** Client disconnect hone par socket.isConnected() true rehta tha aur read() -1 return karta tha, jisse ghost threads infinite loop me 100% CPU consume kar rahe the. Maine bytesRead == -1 par loop break karke finally block me cleanup kiya. Saath hi partial TCP chunks ke liye robust stream parser banaya.

### *Q4: Storage engine me ReentrantReadWriteLock ka kya fayda hai?*

**Ans:** Plain synchronized har operation ko sequentially block karta hai. Redis read-heavy hota hai (80-90% reads). ReentrantReadWriteLock multiple parallel Read Locks allow karta hai, aur sirf SET, INCR ya EXEC commit ke time exclusive Write Lock leta hai. Isse throughput bahut badh jata hai.

### *Q5: Transactions me dirty reads kaise avoid kiye?*

**Ans:** MULTI aane par commands queue hote hain. EXEC par commands pehle ek isolated local scratchpad cache par execute hote hain. Saare commands successfully chalne ke baad ek single atomic step me main store me commit hote hain.

## 4. Ready-to-Copy Resume Bullet Points

• Architected a high-performance in-memory key-value database adhering to the RESP (Redis Serialization Protocol v2) specification using raw Java sockets and Spring Boot.

• Engineered Master-Slave replication engine with 4-way TCP handshake (PING, REPLCONF, PSYNC), binary RDB transfer, continuous write stream propagation, and byte-level offset tracking for synchronous replication via WAIT.

• Designed concurrent in-memory store with Lazy TTL (Time-To-Live) expiration using ConcurrentHashMap and ReentrantReadWriteLock, achieving thread-safe sub-millisecond lookups.

• Implemented atomic transaction pipeline (MULTI, EXEC, DISCARD) with FIFO command queueing and isolated scratchpad execution ensuring strong transactional isolation.

• Authored a robust JUnit 5 test suite consisting of 24 unit and integration tests covering socket stream fragmentation, concurrency, protocol edge cases, and zero-downtime client handling.